Dewetra 3

Release 0.3.14

Dario Rubado

Jun 20, 2024

CONTENTS:

1 dewetra3 package **1**

1.1 Subpackages . . . 1
1.2 Submodules . . . 14
1.3 dewetra3.admin module . . . 14
1.4 dewetra3.asgi module . . . 15
1.5 dewetra3.models module . . . 16
1.6 dewetra3.serializers module . . . 39
1.7 dewetra3.settings module . . . 43
1.8 dewetra3.urls module . . . 43
1.9 dewetra3.views module . . . 44
1.10 dewetra3.wsgi module . . . 49
1.11 Module contents . . . 49

2 manage module **51**

3 version module **53**

4 Indices and tables **55**

Python Module Index **57**

Index **59**

CHAPTER

ONE

DEWETRA3 PACKAGE

1.1 Subpackages

1.1.1 dewetra3.migrations package

Submodules

dewetra3.migrations.0001_initial module

class dewetra3.migrations.0001_initial.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = []

initial = True

```
operations = [<CreateModel name='DraCategory', fields=[('id',
<django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>)], options={'abstract':  False}>, <CreateModel
name='Event', fields=[('id', <django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>)], options={'abstract':  False}>, <CreateModel
name='EventData', fields=[('id', <django.db.models.fields.BigAutoField>), ('date',
<django.db.models.fields.DateField>), ('lat', <django.db.models.fields.FloatField>),
('lon', <django.db.models.fields.FloatField>), ('datatype',
<django.db.models.fields.CharField>), ('event',
<django.db.models.fields.related.ForeignKey>)]>, <CreateModel name='GeoScale',
fields=[('id', <django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>)], options={'abstract':  False}>, <CreateModel
name='Hazard', fields=[('id', <django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>)], options={'abstract':  False}>, <CreateModel
name='Icon', fields=[('id', <django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('value',
<django.db.models.fields.TextField>)], options={'abstract':  False}>, <CreateModel
name='InfoManager', fields=[('id', <django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>), ('component',
<django.db.models.fields.CharField>), ('props',
<django.db.models.fields.json.JSONField>)], options={'abstract':  False}>,
<CreateModel name='InspireTheme', fields=[('id',
<django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>), ('icon',
<django.db.models.fields.related.ForeignKey>)], options={'abstract':  False}>,
<CreateModel name='Layer', fields=[('id', <django.db.models.fields.BigAutoField>),
('name', <django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>), ('dataid',
<django.db.models.fields.CharField>), ('thumb',
<django.db.models.fields.CharField>), ('lonw',
<django.db.models.fields.FloatField>), ('lone',
<django.db.models.fields.FloatField>), ('lats',
<django.db.models.fields.FloatField>), ('latn',
<django.db.models.fields.FloatField>), ('customprops',
<django.db.models.fields.json.JSONField>)], options={'abstract':  False}>,
<CreateModel name='LayerCategory', fields=[('id',
<django.db.models.fields.BigAutoField>), ('name',
<django.db.models.fields.CharField>)], options={'abstract':  False}>, <CreateModel
name='LayerManager', fields=[('id', <django.db.models.fields.BigAutoField>),
('name', <django.db.models.fields.CharField>), ('descr',
<django.db.models.fields.TextField>), ('component',
<django.db.models.fields.CharField>), ('props',
<django.db.models.fields.json.JSONField>)], options={'abstract':  False}>,
<CreateModel name='LegendManager', fields=[('id',
<django.db.models.fields.BigAutoField>), ('descr',
<django.db.models.fields.TextField>), ('component',
<django.db.models.fields.CharField>), ('props',
<django.db.models.fields.json.JSONField>), ('name',
<django.db.models.fields.CharField>)], options={'abstract':  False}>, <CreateModel
name='MetadataManager', fields=[('id', <django.db.models.fields.BigAutoField>),
('descr', <django.db.models.fields.TextField>), ('component',
<django.db.models.fields.CharField>), ('props',
<django.db.models.fields.json.JSONField>), ('name',
<django.db.models.fields.CharField>)], options={'abstract':  False}>, <CreateModel
```

dewetra3.migrations.0002_layer_enabled module

class dewetra3.migrations.0002_layer_enabled.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0001_initial')]

operations = [<AddField model_name='layer', name='enabled', field=<django.db.models.fields.BooleanField>>]

dewetra3.migrations.0003_auto_20240213_1503 module

class dewetra3.migrations.0003_auto_20240213_1503.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0002_layer_enabled')]

operations = [<CreateModel name='Chart', fields=[('id', <django.db.models.fields.BigAutoField>), ('name', <django.db.models.fields.CharField>), ('descr', <django.db.models.fields.TextField>), ('props', <django.db.models.fields.json.JSONField>), ('type', <django.db.models.fields.CharField>)], options={'abstract': False}>, <AddField model_name='server', name='use_proxy', field=<django.db.models.fields.BooleanField>>]

dewetra3.migrations.0004_alter_server_user module

class dewetra3.migrations.0004_alter_server_user.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0003_auto_20240213_1503')]

operations = [<AlterField model_name='server', name='user', field=<django.db.models.fields.CharField>>]

dewetra3.migrations.0005_alter_server_password module

class dewetra3.migrations.0005_alter_server_password.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0004_alter_server_user')]

operations = [<AlterField model_name='server', name='password', field=<django.db.models.fields.CharField>>]

dewetra3.migrations.0006_auto_20240227_0944 module

class dewetra3.migrations.0006_auto_20240227_0944.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0005_alter_server_password')]

operations = [<CreateModel name='ChartManager', fields=[('id', <django.db.models.fields.BigAutoField>), ('name', <django.db.models.fields.CharField>), ('descr', <django.db.models.fields.TextField>), ('component', <django.db.models.fields.CharField>), ('props', <django.db.models.fields.json.JSONField>)], options={'abstract': False}>, <CreateModel name='MapManager', fields=[('id', <django.db.models.fields.BigAutoField>), ('name', <django.db.models.fields.CharField>), ('descr', <django.db.models.fields.TextField>), ('component', <django.db.models.fields.CharField>), ('props', <django.db.models.fields.json.JSONField>)], options={'abstract': False}>, <AddField model_name='layer', name='chartmanager', field=<django.db.models.fields.related.ForeignKey>>, <AddField model_name='layer', name='mapmanager', field=<django.db.models.fields.related.ForeignKey>>]

dewetra3.migrations.0007_tool module

class dewetra3.migrations.0007_tool.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0006_auto_20240227_0944')]

operations = [<CreateModel name='Tool', fields=[('id', <django.db.models.fields.BigAutoField>), ('name', <django.db.models.fields.CharField>), ('descr', <django.db.models.fields.TextField>), ('component', <django.db.models.fields.CharField>), ('props', <django.db.models.fields.json.JSONField>), ('roles', <django.db.models.fields.related.ManyToManyField>)], options={'abstract': False}>]

dewetra3.migrations.0008_tool_icon module

class dewetra3.migrations.0008_tool_icon.**Migration**(*name*, *app_label*)

Bases: Migration

dependencies = [('dewetra3', '0007_tool')]

operations = [<AddField model_name='tool', name='icon', field=<django.db.models.fields.related.ForeignKey>>]

Module contents

1.1.2 dewetra3.dds package

Submodules

dewetra3.dds.views module

class dewetra3.dds.views.**DDSDynamicLayer**(***kwargs*)

Bases: ViewSet

class YourClassName

Bases: object

availabilty(*request*, *layer*)

Retrieves the availability of a layer based on the provided request and layer ID.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer** (*int*) – The ID of the layer.

Returns
The availability of the layer.

Return type
Response

Raises
Response(f'invalid data', status=status.HTTP_500_INTERNAL_SERVER_ERROR) – If the request data is invalid.

basename = None

description = None

detail = None

floodproof(*request*, *layer_id*, *now*)

Retrieves floodproof features for a given layer and time range.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer to retrieve floodproof features for.
- **now** (*int*) – The current timestamp.

Returns
The HTTP response containing the floodproof features.

Return type
Response

floodproofCompatibles(*request*, *layer*)

Retrieves floodproof compatibles for a given layer.

Parameters

- **request** – The HTTP request object.

- **layer** – The ID of the layer.

Returns
A Response object containing the floodproof compatibles.

layerstyle(*request*, *layer*, *layer_id*)

Retrieves the layer style for a given layer ID.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer** (*int*) – The ID of the layer.
- **layer_id** (*int*) – The ID of the layer style.

Returns
A JSON response containing the legend for the layer style.

Return type
Response

list(*request*)

name = None

properties(*request*, *layer_id*)

Retrieve the properties of a layer.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer.

Returns
The response containing the properties of the layer.

Return type
Response

Raises

- *Layer.DoesNotExist* – If the layer with the given ID does not exist.
- **status.HTTP_500_INTERNAL_SERVER_ERROR** – If the properties are invalid or an internal server error occurs.

publish(*request*, *layer*)

Publishes a layer using the provided request and layer ID.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer** (*int*) – The ID of the layer to publish.

Returns
The response containing the published layer ID and type.

Return type
Response

Raises
Response(status=status.HTTP_500_INTERNAL_SERVER_ERROR) – If the data is invalid.

publish_default(*request*, *layer_id*)

Publishes the default layer for a given layer ID.

Parameters

- **request** – The HTTP request object.
- **layer_id** – The ID of the layer to publish.

Returns
A Response object containing the server URL and the published layer ID.

Raises

- **HTTP_500_INTERNAL_SERVER_ERROR** – If the layer properties or attributes are invalid.
- **HTTP_404_NOT_FOUND** – If the data is not found.

punctualserie(*request*, *layer*)

Retrieve punctual series data from the DDS client.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer** (*int*) – The ID of the layer to retrieve data from.

Returns
The HTTP response containing the punctual series data.

Return type
Response

republish(*request*, *layer_id*)

Republishes a layer with the given layer_id using the provided request.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer to republish.

Returns
The HTTP response containing the republishing result.

Return type
Response

Raises

- *Layer.DoesNotExist* – If the layer with the given layer_id does not exist.
- **status.HTTP_500_INTERNAL_SERVER_ERROR** – If the provided data is invalid.

suffix = None

variables(*request*, *layer_id*)

Retrieves the variables associated with a layer.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer.

Returns
The response containing the variables associated with the layer.

Return type
Response

Module contents

1.1.3 dewetra3.sentinel package

Submodules

dewetra3.sentinel.views module

class dewetra3.sentinel.views.**SentinelLayer**(***kwargs*)

Bases: ViewSet

This class provides the following functionalities:

- sensors: returns the list of sensors of a given class
- aggregations: returns the list of aggregations
- aggregated_layer: returns the aggregated layer of a given class
- sensor_info: returns the info of a given sensor
- stations_sensors: returns the list of sensors of a given station

aggregated_layer(*request*, *sensor_class*)

Retrieves aggregated sensor data based on the provided parameters.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **sensor_class** (*str*) – The sensor class to filter the data.

Returns
The HTTP response object containing the aggregated sensor data.

Return type
Response

Raises
ValueError – If the 'aggr_layer' parameter is missing in the request.

aggregations(*request*)

Retrieves the aggregation layers from the Sentinel service.

Parameters
request – The HTTP request object.

Returns
A Response object containing the aggregation layers retrieved from the Sentinel service.

basename = None

description = None

detail = None

name = None

sensor_info(*request*, *sensor_id*, *db_id*)

Retrieves information about a sensor.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **sensor_id** (*int*) – The ID of the sensor.
- **db_id** (*int*) – The ID of the database.

Returns
The response containing the sensor information.

Return type
Response

sensors(*request*, *sensor_class*)

Retrieves sensor data based on the provided parameters.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **sensor_class** (*str*) – The class of the sensor.

Returns
The HTTP response containing the sensor data.

Return type
Response

Raises
N/A –

stations_sensors(*request*, *station_id*, *db_id*)

Retrieve information about sensors for a specific station.

Parameters

- **request** – The HTTP request object.
- **station_id** – The ID of the station.
- **db_id** – The ID of the database.

Returns
A Response object containing the information about sensors for the specified station.

suffix = None

Module contents

1.1.4 dewetra3.tools package

Submodules

dewetra3.tools.views module

class dewetra3.tools.views.**ToolsViewsSet**(***kwargs*)

Bases: ViewSet

A viewset for handling tools-related operations.

basename = None

description = None

detail = None

name = None

print(*request*, *print_type*)

Prints the request data using the specified print type.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **print_type** (*str*) – The type of print (pdf or image).

Returns
The response from the print service.

Return type
HttpResponse

suffix = None

upload_image(*request*)

Uploads an image file.

Parameters
request (*HttpRequest*) – The HTTP request object.

Returns
The response containing the file path.

Return type
HttpResponse

Module contents

1.1.5 dewetra3.charts package

Submodules

dewetra3.charts.views module

class dewetra3.charts.views.**ChartViewSet**(***kwargs*)

Bases: ViewSet

A viewset for handling chart build operations.

basename = None

dds_chart_image(*request*, *chart_id*)

Generate and return a chart image based on the provided chart ID and request data.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **chart_id** (*str*) – The ID of the chart to generate the image for.

Returns
The HTTP response object containing the generated chart image.

Return type
Response

Raises
None –

dds_chart_json(*request*)

Generate a JSON representation of a chart based on the given request.

Parameters
request (*HttpRequest*) – The HTTP request object.

Returns
The HTTP response object containing the JSON representation of the chart.

Return type
Response

dds_punctual_chart_image(*request*, *layer_id*)

Generate and return a punctual chart image.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer.

Returns
The HTTP response object containing the chart image.

Return type
Response

dds_punctual_chart_json(*request*, *layer_id*)

Generate a JSON representation of a punctual chart.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **layer_id** (*int*) – The ID of the layer.

Returns
The HTTP response containing the JSON representation of the chart.

Return type
Response

description = None

detail = None

name = None

suffix = None

Module contents

1.1.6 dewetra3.test package

Module contents

1.1.7 dewetra3.widget package

Submodules

dewetra3.widget.service module

Created on May 14, 2020

@author: doy

class dewetra3.widget.service.**WidgetService**

Bases: Thread

cbe monitoring class

on_close(*ws*)

on_error(*ws*, *error*)

on_message(*ws*, *message*)

run()

Method representing the thread's activity.

You may override this method in a subclass. The standard run() method invokes the callable object passed to the object's constructor as the target argument, if any, with sequential and keyword arguments taken from the args and kwargs arguments, respectively.

dewetra3.widget.tests module

dewetra3.widget.views module

class dewetra3.widget.views.**WidgetDataView**(***kwargs*)

Bases: ViewSet

A viewset for handling widget data requests.

basename = None

description = None

detail = None

name = None

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

suffix = None

widget_data(*request*, *widget_name*)

Retrieve data for a specific widget.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **widget_name** (*str*) – The name of the widget.

Returns
The response containing the widget data.

Return type
Response

widget_img(*request*, *widget_name*, *id*)

Return the image for a specific widget.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **widget_name** (*str*) – The name of the widget.
- **id** (*str*) – The ID of the image.

Returns
The HTTP response containing the image.

Return type
HttpResponse

dewetra3.widget.widget module

class dewetra3.widget.widget.**WidgetManager**(*widget*)

Bases: object

get_data()

Retrieves data for the widget.

Returns
A list of dictionaries containing the widget data.

get_img(*id*)

Retrieve the image file path associated with the given ID.

Parameters
id (*int*) – The ID of the image to retrieve.

Returns
The file path of the image if found, None otherwise.

Return type
str or None

update()

Update the widget by retrieving properties and availability data from the DDS server.

This method performs the following steps: 1. Retrieves the DDS client for the widget's layer server. 2. Gets and updates the properties of the widget. 3. Retrieves the availability data for the specified time range. 4. Publishes the layer by downloading the image(s) based on the widget's mode.

Raises

Exception – If an error occurs while updating the widget.

Module contents

1.2 Submodules

1.3 dewetra3.admin module

class dewetra3.admin.**AppAdmin**(*model*, *admin_site*)

Bases: ModelAdmin

formfield_overrides = {<class 'django.db.models.fields.json.JSONField'>: {'widget': <class 'django_json_widget.widgets.JSONEditorWidget'>}}

list_max_show_all = 1000

list_per_page = 200

property media

save_as = True

search_fields = ['name']

class dewetra3.admin.**FilterByCategory**(*request*, *params*, *model*, *model_admin*)

Bases: SimpleListFilter

lookups(*request*, *model_admin*)

Must be overridden to return a list of tuples (value, verbose value)

parameter_name = 'category'

queryset(*request*, *queryset*)

Return the filtered queryset.

title = 'Category'

class dewetra3.admin.**FilterByLayerManager**(*request*, *params*, *model*, *model_admin*)

Bases: SimpleListFilter

lookups(*request*, *model_admin*)

Must be overridden to return a list of tuples (value, verbose value)

parameter_name = 'layermanager'

queryset(*request*, *queryset*)

Return the filtered queryset.

title = 'Layer Manager'

class dewetra3.admin.**IsEnabledFilter**(*request*, *params*, *model*, *model_admin*)

Bases: SimpleListFilter

default_value()

lookups(*request*, *model_admin*)

Must be overridden to return a list of tuples (value, verbose value)

parameter_name = 'enabled'

queryset(*request*, *queryset*)

Return the filtered queryset.

title = 'Is Enabled'

class dewetra3.admin.**LayerAdmin**(*model*, *admin_site*)

Bases: ModelAdmin

custom_name(*obj:* Layer)

formfield_overrides = {<class 'django.db.models.fields.json.JSONField'>: {'widget': <class 'django_json_widget.widgets.JSONEditorWidget'>}}

get_queryset(*request*)

Return a QuerySet of all model instances that can be edited by the admin site. This is used by changelist_view.

list_display = ['custom_name', 'enabled']

list_filter = [<class 'dewetra3.admin.IsEnabledFilter'>, <class 'dewetra3.admin.FilterByCategory'>, <class 'dewetra3.admin.FilterByLayerManager'>]

list_max_show_all = 1000

list_per_page = 200

property media

save_as = True

search_fields = ['name']

1.4 dewetra3.asgi module

ASGI config for dewetra3 project.

It exposes the ASGI callable as a module-level variable named `application`.

For more information on this file, see https://docs.djangoproject.com/en/3.2/howto/deployment/asgi/

1.5 dewetra3.models module

class dewetra3.models.**AbstractDescrModel**(**args*, ***kwargs*)

Bases: *AbstractNameModel*

class Meta

Bases: object

abstract = False

descr

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**AbstractIconDescrModel**(**args*, ***kwargs*)

Bases: *AbstractDescrModel*

class Meta

Bases: object

abstract = False

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

icon_id

class dewetra3.models.**AbstractNameModel**(**args*, ***kwargs*)

Bases: Model

class Meta

Bases: object

abstract = False

name

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**AbstractWithComponentModel**(**args*, ***kwargs*)

Bases: *AbstractDescrModel*

class Meta

Bases: object

abstract = False

component

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

props

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**AbstractWithPropertiesModel**(**args*, ***kwargs*)

Bases: *AbstractDescrModel*

class Meta

Bases: object

abstract = False

props

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**Chart**(*id*, *name*, *descr*, *props*, *type*)

Bases: *AbstractWithPropertiesModel*

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

type

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**ChartManager**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the visualization of the charts coming from the layer

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**DraCategory**(*id*, *name*, *descr*, *icon*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

Pizza.toppings and Topping.pizzas are ManyToManyDescriptor instances.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Event**(*id*, *name*, *descr*)

Bases: *AbstractDescrModel*

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

eventdata_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**EventData**(*id*, *event*, *date*, *lat*, *lon*, *datatype*, *hazard*)

Bases: `Model`

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

datatype

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

date

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

event

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

event_id

get_next_by_date(*, *field=<django.db.models.fields.DateField: date>*, *is_next=True*, ***kwargs*)

get_previous_by_date(*, *field=<django.db.models.fields.DateField: date>*, *is_next=False*, ***kwargs*)

hazard

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

hazard_id

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

lat

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

lon

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**GeoScale**(*id*, *name*, *descr*)

Bases: *AbstractDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Hazard**(*id*, *name*, *descr*, *icon*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

eventdata_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Icon**(*id*, *name*, *value*)

Bases: *AbstractNameModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

dracategory_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

hazard_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

inspiretheme_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

source_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

tag_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

tool_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

value

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class dewetra3.models.**InfoManager**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the information coming from the layer

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**InspireTheme**(*id*, *name*, *descr*, *icon*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Layer**(*id*, *name*, *descr*, *dataid*, *server*, *thumb*, *icon*, *source*, *lonw*, *lone*, *lats*, *latn*, *enabled*, *category*, *event*, *geoscale*, *infomanager*, *legendmanager*, *propertiesmanager*, *metadatamanager*, *layermanager*, *mapmanager*, *chartmanager*, *customprops*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

category

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

category_id

chartmanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

chartmanager_id

customprops

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dataid

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

dracategories

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

enabled

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

event

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

event_id

geoscale

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

geoscale_id

hazards

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

infomanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

infomanager_id

inspirethemes

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

latn

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

lats

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layermanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

layermanager_id

legendmanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

legendmanager_id

lone

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

lonw

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

mapmanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

mapmanager_id

metadatamanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

metadatamanager_id

objects = <django.db.models.manager.Manager object>

paths

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

propertiesmanager

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

propertiesmanager_id

related

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

roles

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

server

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

server_id

source

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

source_id

tags

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

thumb

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

widget_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

class dewetra3.models.**LayerCategory**(*id*, *name*)

Bases: *AbstractNameModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layers

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**LayerManager**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the data from the layer

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**LegendManager**(*id*, *descr*, *component*, *props*, *name*)

Bases: *AbstractWithComponentModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**MapManager**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the visualization of the layer on the map

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**MetadataManager**(*id*, *descr*, *component*, *props*, *name*)

Bases: *AbstractWithComponentModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Path**(*id*, *name*)

Bases: *AbstractNameModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

hierarchy

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**PathHierarchy**(*id*, *path*, *hierarchy*)

Bases: `Model`

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

hierarchy

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

path

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

path_id

class dewetra3.models.**PropertiesManager**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the properties of the layer

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Parent.children is a ReverseManyToOneDescriptor instance.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Role**(*id*, *name*)

Bases: *AbstractNameModel*

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layers

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

Pizza.toppings and Topping.pizzas are ManyToManyDescriptor instances.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

objects = <django.db.models.manager.Manager object>

series

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

tools

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

class dewetra3.models.**Serie**(*id*, *name*)

Bases: *AbstractNameModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

roles

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

class dewetra3.models.**Server**(*id*, *name*, *descr*, *url*, *user*, *password*, *use_proxy*)

Bases: *AbstractDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

password

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

url

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

use_proxy

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

user

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

class `dewetra3.models.`**Source**(*id*, *name*, *descr*, *icon*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the reverse side of a many-to-one relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Parent.children` is a `ReverseManyToOneDescriptor` instance.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Tag**(*id*, *name*, *descr*, *icon*)

Bases: *AbstractIconDescrModel*

exception DoesNotExist

Bases: `ObjectDoesNotExist`

exception MultipleObjectsReturned

Bases: `MultipleObjectsReturned`

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer_set

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

`Pizza.toppings` and `Topping.pizzas` are `ManyToManyDescriptor` instances.

Most of the implementation is delegated to a dynamically defined manager class built by `create_forward_many_to_many_manager()` defined below.

objects = <django.db.models.manager.Manager object>

class dewetra3.models.**Tool**(**args*, ***kwargs*)

Bases: *AbstractWithComponentModel*

This class is used to manage the tools that are available in the application

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

icon

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

Child.parent is a ForwardManyToOneDescriptor instance.

icon_id

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

roles

Accessor to the related objects manager on the forward and reverse sides of a many-to-many relation.

In the example:

```
class Pizza(Model):
    toppings = ManyToManyField(Topping, related_name='pizzas')
```

Pizza.toppings and Topping.pizzas are ManyToManyDescriptor instances.

Most of the implementation is delegated to a dynamically defined manager class built by create_forward_many_to_many_manager() defined below.

class dewetra3.models.**Widget**(*id*, *name*, *layer*, *mode*, *wmsparams*, *wmsbaselayer*, *alwaysupdate*, *period*, *properties*)

Bases: Model

exception DoesNotExist

Bases: ObjectDoesNotExist

exception MultipleObjectsReturned

Bases: MultipleObjectsReturned

alwaysupdate

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

id

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

layer

Accessor to the related object on the forward side of a many-to-one or one-to-one (via ForwardOneToOneDescriptor subclass) relation.

In the example:

```
class Child(Model):
    parent = ForeignKey(Parent, related_name='children')
```

`Child.parent` is a `ForwardManyToOneDescriptor` instance.

layer_id

mode

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

name

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

objects = <django.db.models.manager.Manager object>

period

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

properties

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

wmsbaselayer

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

wmsparams

A wrapper for a deferred-loading field. When the value is read from this object the first time, the query is executed.

1.6 dewetra3.serializers module

class dewetra3.serializers.**AbstractComponentSerializer**(**args*, ***kwargs*)

Bases: `ModelSerializer`

class Meta

Bases: `object`

abstract = True

fields = ['id', 'component', 'props']

class dewetra3.serializers.**AbstractNameIconSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

abstract = True

fields = ['id', 'name', 'icon']

class dewetra3.serializers.**ChartManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *ChartManager*

class dewetra3.serializers.**DraCategorySerializer**(**args*, ***kwargs*)

Bases: *AbstractNameIconSerializer*

class Meta

Bases: *Meta*

model

alias of *DraCategory*

class dewetra3.serializers.**EventDataSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = '__all__'

model

alias of *EventData*

class dewetra3.serializers.**EventSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

exclude = ('id',)

model

alias of *Event*

class dewetra3.serializers.**GeoScaleSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = '__all__'

model

alias of *GeoScale*

class dewetra3.serializers.**HazardSerializer**(**args*, ***kwargs*)

Bases: *AbstractNameIconSerializer*

class Meta

Bases: *Meta*

model

alias of *Hazard*

class dewetra3.serializers.**InfoManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *InfoManager*

class dewetra3.serializers.**IspireThemeSerializer**(**args*, ***kwargs*)

Bases: *AbstractNameIconSerializer*

class Meta

Bases: *Meta*

model

alias of *InspireTheme*

class dewetra3.serializers.**LayerCategorySerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = ['name']

model

alias of *LayerCategory*

class dewetra3.serializers.**LayerManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *LayerManager*

class dewetra3.serializers.**LayerSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = '__all__'

model

alias of *Layer*

class dewetra3.serializers.**LegendManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *LegendManager*

class dewetra3.serializers.**MapManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *MapManager*

class dewetra3.serializers.**MetadataManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *MetadataManager*

class dewetra3.serializers.**PathHierarchySerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = '__all__'

model

alias of *PathHierarchy*

class dewetra3.serializers.**PropertiesManagerSerializer**(**args*, ***kwargs*)

Bases: *AbstractComponentSerializer*

class Meta

Bases: *Meta*

model

alias of *PropertiesManager*

class dewetra3.serializers.**ServerSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

exclude = ('user', 'password', 'use_proxy')

model

alias of *Server*

get_url(*obj*)

class dewetra3.serializers.**SourceSerializer**(**args*, ***kwargs*)

Bases: *AbstractNameIconSerializer*

class Meta

Bases: *Meta*

model

alias of *Source*

class dewetra3.serializers.**TagSerializer**(**args*, ***kwargs*)

Bases: *AbstractNameIconSerializer*

class Meta

Bases: *Meta*

model

alias of *Tag*

class dewetra3.serializers.**ToolSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

fields = '__all__'

model

alias of *Tool*

1.7 dewetra3.settings module

Django settings for dewetra3 project.

Generated by 'django-admin startproject' using Django 3.2.5.

For more information on this file, see https://docs.djangoproject.com/en/3.2/topics/settings/

For the full list of settings and their values, see https://docs.djangoproject.com/en/3.2/ref/settings/

1.8 dewetra3.urls module

dewetra3 URL Configuration

The *urlpatterns* list routes URLs to views. For more information please see:
https://docs.djangoproject.com/en/3.2/topics/http/urls/

Examples: Function views

1. Add an import: from my_app import views
2. Add a URL to urlpatterns: path('', views.home, name='home')

Class-based views

1. Add an import: from other_app.views import Home
2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')

Including another URLconf

1. Import the include() function: from django.urls import include, path
2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))

1.9 dewetra3.views module

class dewetra3.views.**BypassFormatArgumentNegotiation**

Bases: `BaseContentNegotiation`

A content negotiation class that ignores the 'format' argument.

This class is used to bypass the format argument negotiation in content negotiation. It always selects the first parser and renderer from the available options.

select_parser(*request*, *parsers*)

select_renderer(*request*, *renderers*, *format_suffix*)

class dewetra3.views.**CapabilitiesView**(***kwargs*)

Bases: `APIView`

GetCapabilities proxy view with json translation.

This view handles the GET and OPTIONS requests for retrieving capabilities. It acts as a proxy to the *capabilities* function, passing the request, primary key (pk), and format as parameters.

content_negotiation_class

The content negotiation class to use.

Type
class

get(*request*, *pk*, *format=None*)

Handles the GET request.

options(*request*, *pk*, *format=None*)

Handles the OPTIONS request.

content_negotiation_class

alias of `BypassFormatArgumentNegotiation`

get(*request*, *pk*, *format=None*)

options(*request*, *pk*, *format=None*)

Handler method for HTTP 'OPTIONS' request.

class dewetra3.views.**LayerCategoryView**(***kwargs*)

Bases: `ReadOnlyModelViewSet`

A viewset for retrieving LayerCategory objects.

This viewset provides read-only endpoints for retrieving LayerCategory objects.

basename = None

description = None

detail = None

name = None

queryset = <QuerySet [<LayerCategory: observation>, <LayerCategory: event>, <LayerCategory: static>, <LayerCategory: forecast>]>

serializer_class

alias of *LayerCategorySerializer*

suffix = None

class dewetra3.views.**LayerView**(***kwargs*)

Bases: ReadOnlyModelViewSet

A viewset for retrieving read-only Layer objects.

This viewset provides the following actions: - list: Retrieves a list of Layer objects. - retrieve: Retrieves a specific Layer object by its ID.

Additional filtering can be applied by specifying the 'category' query parameter.

Only authenticated users are allowed to access this viewset.

basename = None

description = None

detail = None

get_queryset()

Returns the queryset of Layer objects.

The queryset is filtered based on the 'category' query parameter and the user's domain.

Returns
The filtered queryset of Layer objects.

Return type
QuerySet

name = None

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

queryset = <QuerySet []>

serializer_class

alias of *LayerSerializer*

suffix = None

class dewetra3.views.**ProxyView**(***kwargs*)

Bases: APIView

Custom view to proxy requests to the server.

content_negotiation_class

alias of *BypassFormatArgumentNegotiation*

get(*request*, *pk*, *path*)

Handles GET requests to proxy data to the server.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **pk** (*int*) – The primary key of the object.
- **path** (*str*) – The path to proxy the request to.

Returns
The response from the server.

Return type
HttpResponse

options(*request*, *pk*, *path*)

Handles OPTIONS requests to proxy data to the server.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **pk** (*int*) – The primary key of the object.
- **path** (*str*) – The path to proxy the request to.

Returns
The response from the server.

Return type
HttpResponse

post(*request*, *pk*, *path*)

Handles POST requests to proxy data to the server.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **pk** (*int*) – The primary key of the object.
- **path** (*str*) – The path to proxy the request to.

Returns
The response from the server.

Return type
HttpResponse

class dewetra3.views.**ServerView**(***kwargs*)

Bases: ReadOnlyModelViewSet

A viewset for retrieving server data.

This viewset provides read-only endpoints for retrieving server objects.

basename = None

description = None

detail = None

name = None

queryset = <QuerySet [<Server: GeoServer CimaFoundation>, <Server: Italia Sicura>, <Server: Geoserver geonode>, <Server: Dewetra Data Server at Protezione Civile Marche N>, <Server: SIT-DPC [IDROGEOLOGICO]>, <Server: Geonode myDEWETRAworld>, <Server: Ispra PAI frane>, <Server: SIT-DPC [ISTAT]>, <Server: SIT-DPC [TRASPORTI]>, <Server: SIT-DPC [CARTOGRAFIA]>, <Server: GSRISICO>, <Server: erra_GEOSERVER>, <Server: bb01 dds>, <Server: proxysafe>, <Server: SIT-DPC [SANITARIO]>, <Server: Rasor>, <Server: geoserver_test>, <Server: Dewetra Data Server at Cima Foundation-OLD>, <Server: Flood Obeservatory colorado NA_2wk_rs>, <Server: Flood Observatory NA_3day_rs>, '...(remaining elements truncated)...']>

serializer_class

alias of *ServerSerializer*

suffix = None

class dewetra3.views.**TagView**(***kwargs*)

Bases: ReadOnlyModelViewSet

A viewset for retrieving Tag objects.

This viewset provides read-only endpoints for retrieving Tag objects.

basename = None

description = None

detail = None

name = None

queryset = <QuerySet [<Tag: Temporali>, <Tag: Pioggia>, <Tag: E_Flooded_Areas>, <Tag: Pericolosità Idraulica - GAR 2015>, <Tag: Modelli Meteo>, <Tag: Modelli Neve>, <Tag: Aree a Rischio>, <Tag: Corpi idrici - Aree allagate>, <Tag: Esposti>, <Tag: Idrografia>, <Tag: Alluvioni - Aree Inondate>, <Tag: E_Grading>, <Tag: Incendi>, <Tag: Neve>, <Tag: Temperatura>, <Tag: Trasporti>, <Tag: Modelli Idro>, <Tag: Hazard>, <Tag: Flooded Areas>, <Tag: Drought>, '...(remaining elements truncated)...']>

serializer_class

alias of *TagSerializer*

suffix = None

class dewetra3.views.**ToolView**(***kwargs*)

Bases: ReadOnlyModelViewSet

A viewset for retrieving tools based on user domain.

queryset

The queryset of Tool objects.

Type
QuerySet

serializer_class

The serializer class for Tool objects.

Type
Serializer

basename = None

description = None

detail = None

get_queryset()

Get the list of items for this view. This must be an iterable, and may be a queryset. Defaults to using *self.queryset*.

This method should always be used rather than accessing *self.queryset* directly, as *self.queryset* gets evaluated only once, and those results are cached for all subsequent requests.

You may want to override this if you need to provide different querysets depending on the incoming request.

(Eg. return a list of items that is specific to the user)

name = None

queryset = <QuerySet [<Tool: UploadToGeoserver (UploaderComponent) >]>

serializer_class

alias of *ToolSerializer*

suffix = None

dewetra3.views.**capabilities**(*request*, *pk=None*, *format=None*)

Get the capabilities of the server and return them as json.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **pk** (*int, optional*) – The ID of the server. Defaults to None.
- **format** (*str, optional*) – The format of the response. Defaults to None.

Returns
The capabilities of the server as a JSON response.

Return type
Response

dewetra3.views.**proxy**(*request*, *pk=None*, *path=None*, *format=None*)

Proxies the request to a server specified by the given primary key (pk) and path. The function forwards the request to the server and returns the response to the client.

Parameters

- **request** (*HttpRequest*) – The incoming request object.
- **pk** (*int*) – The primary key of the server object to proxy the request to.
- **path** (*str*) – The path to append to the server's URL.
- **format** (*str*) – The desired response format.

Returns
The response from the server.

Return type
HttpResponse

Raises
Server.DoesNotExist – If the server with the given primary key does not exist.

1.10 dewetra3.wsgi module

WSGI config for dewetra3 project.

It exposes the WSGI callable as a module-level variable named `application`.

For more information on this file, see https://docs.djangoproject.com/en/3.2/howto/deployment/wsgi/

1.11 Module contents

CHAPTER

TWO

MANAGE MODULE

Django's command-line utility for administrative tasks.

`manage.main()`

Run administrative tasks.

CHAPTER

THREE

VERSION MODULE

CHAPTER

FOUR

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

d

dewetra3, 49
dewetra3.admin, 14
dewetra3.asgi, 15
dewetra3.charts, 12
dewetra3.charts.views, 10
dewetra3.dds, 8
dewetra3.dds.views, 5
dewetra3.migrations, 5
dewetra3.migrations.0001_initial, 1
dewetra3.migrations.0002_layer_enabled, 3
dewetra3.migrations.0003_auto_20240213_1503, 3
dewetra3.migrations.0004_alter_server_user, 3
dewetra3.migrations.0005_alter_server_password, 3
dewetra3.migrations.0006_auto_20240227_0944, 4
dewetra3.migrations.0007_tool, 4
dewetra3.migrations.0008_tool_icon, 4
dewetra3.models, 16
dewetra3.sentinel, 9
dewetra3.sentinel.views, 8
dewetra3.serializers, 39
dewetra3.settings, 43
dewetra3.test, 12
dewetra3.tools, 10
dewetra3.tools.views, 9
dewetra3.urls, 43
dewetra3.views, 44
dewetra3.widget, 14
dewetra3.widget.service, 12
dewetra3.widget.tests, 12
dewetra3.widget.views, 12
dewetra3.widget.widget, 13
dewetra3.wsgi, 49

m

manage, 51

v

version, 53

INDEX

A

abstract (*dewetra3.models.AbstractDescrModel.Meta attribute*), 16
abstract (*dewetra3.models.AbstractIconDescrModel.Meta attribute*), 16
abstract (*dewetra3.models.AbstractNameModel.Meta attribute*), 16
abstract (*dewetra3.models.AbstractWithComponentModel.Meta attribute*), 16
abstract (*dewetra3.models.AbstractWithPropertiesModel.Meta attribute*), 17
abstract (*dewetra3.serializers.AbstractComponentSerializer.Meta attribute*), 39
abstract (*dewetra3.serializers.AbstractNameIconSerializer.Meta attribute*), 40
AbstractComponentSerializer (*class in dewetra3.serializers*), 39
AbstractComponentSerializer.Meta (*class in dewetra3.serializers*), 39
AbstractDescrModel (*class in dewetra3.models*), 16
AbstractDescrModel.Meta (*class in dewetra3.models*), 16
AbstractIconDescrModel (*class in dewetra3.models*), 16
AbstractIconDescrModel.Meta (*class in dewetra3.models*), 16
AbstractNameIconSerializer (*class in dewetra3.serializers*), 39
AbstractNameIconSerializer.Meta (*class in dewetra3.serializers*), 40
AbstractNameModel (*class in dewetra3.models*), 16
AbstractNameModel.Meta (*class in dewetra3.models*), 16
AbstractWithComponentModel (*class in dewetra3.models*), 16
AbstractWithComponentModel.Meta (*class in dewetra3.models*), 16
AbstractWithPropertiesModel (*class in dewetra3.models*), 17
AbstractWithPropertiesModel.Meta (*class in dewetra3.models*), 17
aggregated_layer() (*dewetra3.sentinel.views.SentinelLayer method*), 8
aggregations() (*dewetra3.sentinel.views.SentinelLayer method*), 8
alwaysupdate (*dewetra3.models.Widget attribute*), 38
AppAdmin (*class in dewetra3.admin*), 14
availabilty() (*dewetra3.dds.views.DDSDynamicLayer method*), 5

B

basename (*dewetra3.charts.views.ChartViewSet attribute*), 10
basename (*dewetra3.dds.views.DDSDynamicLayer attribute*), 5
basename (*dewetra3.sentinel.views.SentinelLayer attribute*), 8
basename (*dewetra3.tools.views.ToolsViewsSet attribute*), 10
basename (*dewetra3.views.LayerCategoryView attribute*), 44
basename (*dewetra3.views.LayerView attribute*), 45
basename (*dewetra3.views.ServerView attribute*), 46
basename (*dewetra3.views.TagView attribute*), 47
basename (*dewetra3.views.ToolView attribute*), 48
basename (*dewetra3.widget.views.WidgetDataView attribute*), 12
BypassFormatArgumentNegotiation (*class in dewetra3.views*), 44

C

capabilities() (*in module dewetra3.views*), 48
CapabilitiesView (*class in dewetra3.views*), 44
category (*dewetra3.models.Layer attribute*), 25
category_id (*dewetra3.models.Layer attribute*), 25
Chart (*class in dewetra3.models*), 17
Chart.DoesNotExist, 17
Chart.MultipleObjectsReturned, 17
ChartManager (*class in dewetra3.models*), 17
chartmanager (*dewetra3.models.Layer attribute*), 25
ChartManager.DoesNotExist, 17

ChartManager.MultipleObjectsReturned, 17
chartmanager_id (*dewetra3.models.Layer attribute*), 25
ChartManagerSerializer (*class in dewetra3.serializers*), 40
ChartManagerSerializer.Meta (*class in dewetra3.serializers*), 40
ChartViewSet (*class in dewetra3.charts.views*), 10
component (*dewetra3.models.AbstractWithComponentModel attribute*), 16
content_negotiation_class (*dewetra3.views.CapabilitiesView attribute*), 44
content_negotiation_class (*dewetra3.views.ProxyView attribute*), 45
custom_name() (*dewetra3.admin.LayerAdmin method*), 15
customprops (*dewetra3.models.Layer attribute*), 25

D

dataid (*dewetra3.models.Layer attribute*), 25
datatype (*dewetra3.models.EventData attribute*), 19
date (*dewetra3.models.EventData attribute*), 19
dds_chart_image() (*dewetra3.charts.views.ChartViewSet method*), 10
dds_chart_json() (*dewetra3.charts.views.ChartViewSet method*), 11
dds_punctual_chart_image() (*dewetra3.charts.views.ChartViewSet method*), 11
dds_punctual_chart_json() (*dewetra3.charts.views.ChartViewSet method*), 11
DDSDynamicLayer (*class in dewetra3.dds.views*), 5
DDSDynamicLayer.YourClassName (*class in dewetra3.dds.views*), 5
default_value() (*dewetra3.admin.IsEnabledFilter method*), 15
dependencies (*dewetra3.migrations.0001_initial.Migration attribute*), 1
dependencies (*dewetra3.migrations.0002_layer_enabled.Migration attribute*), 3
dependencies (*dewetra3.migrations.0003_auto_20240213_1505.Migration attribute*), 3
dependencies (*dewetra3.migrations.0004_alter_server_user.Migration attribute*), 3
dependencies (*dewetra3.migrations.0005_alter_server_password.Migration attribute*), 3
dependencies (*dewetra3.migrations.0006_auto_20240227_0944.Migration attribute*), 4
dependencies (*dewetra3.migrations.0007_tool.Migration attribute*), 4
dependencies (*dewetra3.migrations.0008_tool_icon.Migration attribute*), 4
descr (*dewetra3.models.AbstractDescrModel attribute*), 16
description (*dewetra3.charts.views.ChartViewSet attribute*), 11
description (*dewetra3.dds.views.DDSDynamicLayer attribute*), 5
description (*dewetra3.sentinel.views.SentinelLayer attribute*), 8
description (*dewetra3.tools.views.ToolsViewsSet attribute*), 10
description (*dewetra3.views.LayerCategoryView attribute*), 45
description (*dewetra3.views.LayerView attribute*), 45
description (*dewetra3.views.ServerView attribute*), 46
description (*dewetra3.views.TagView attribute*), 47
description (*dewetra3.views.ToolView attribute*), 48
description (*dewetra3.widget.views.WidgetDataView attribute*), 12
detail (*dewetra3.charts.views.ChartViewSet attribute*), 11
detail (*dewetra3.dds.views.DDSDynamicLayer attribute*), 5
detail (*dewetra3.sentinel.views.SentinelLayer attribute*), 8
detail (*dewetra3.tools.views.ToolsViewsSet attribute*), 10
detail (*dewetra3.views.LayerCategoryView attribute*), 45
detail (*dewetra3.views.LayerView attribute*), 45
detail (*dewetra3.views.ServerView attribute*), 46
detail (*dewetra3.views.TagView attribute*), 47
detail (*dewetra3.views.ToolView attribute*), 48
detail (*dewetra3.widget.views.WidgetDataView attribute*), 12
dewetra3
    module, 49
dewetra3.admin
    module, 14
dewetra3.asgi
    module, 15
dewetra3.charts
    module, 12
dewetra3.charts.views
    module, 10
dewetra3.dds
    module, 8
dewetra3.dds.views
    module, 5
dewetra3.migrations
    module, 5
dewetra3.migrations.0001_initial
    module, 1

dewetra3.migrations.0002_layer_enabled
module, 3
dewetra3.migrations.0003_auto_20240213_1503
module, 3
dewetra3.migrations.0004_alter_server_user
module, 3
dewetra3.migrations.0005_alter_server_password
module, 3
dewetra3.migrations.0006_auto_20240227_0944
module, 4
dewetra3.migrations.0007_tool
module, 4
dewetra3.migrations.0008_tool_icon
module, 4
dewetra3.models
module, 16
dewetra3.sentinel
module, 9
dewetra3.sentinel.views
module, 8
dewetra3.serializers
module, 39
dewetra3.settings
module, 43
dewetra3.test
module, 12
dewetra3.tools
module, 10
dewetra3.tools.views
module, 9
dewetra3.urls
module, 43
dewetra3.views
module, 44
dewetra3.widget
module, 14
dewetra3.widget.service
module, 12
dewetra3.widget.tests
module, 12
dewetra3.widget.views
module, 12
dewetra3.widget.widget
module, 13
dewetra3.wsgi
module, 49
dracategories (*dewetra3.models.Layer attribute*), 25
DraCategory (*class in dewetra3.models*), 18
DraCategory.DoesNotExist, 18
DraCategory.MultipleObjectsReturned, 18
dracategory_set (*dewetra3.models.Icon attribute*), 21
DraCategorySerializer (*class in dewetra3.serializers*), 40
DraCategorySerializer.Meta (*class in dewetra3.serializers*), 40

E

enabled (*dewetra3.models.Layer attribute*), 25
Event (*class in dewetra3.models*), 18
event (*dewetra3.models.EventData attribute*), 19
event (*dewetra3.models.Layer attribute*), 25
Event.DoesNotExist, 18
Event.MultipleObjectsReturned, 18
event_id (*dewetra3.models.EventData attribute*), 19
event_id (*dewetra3.models.Layer attribute*), 26
EventData (*class in dewetra3.models*), 19
EventData.DoesNotExist, 19
EventData.MultipleObjectsReturned, 19
eventdata_set (*dewetra3.models.Event attribute*), 18
eventdata_set (*dewetra3.models.Hazard attribute*), 21
EventDataSerializer (*class in dewetra3.serializers*), 40
EventDataSerializer.Meta (*class in dewetra3.serializers*), 40
EventSerializer (*class in dewetra3.serializers*), 40
EventSerializer.Meta (*class in dewetra3.serializers*), 40
exclude (*dewetra3.serializers.EventSerializer.Meta attribute*), 40
exclude (*dewetra3.serializers.ServerSerializer.Meta attribute*), 42

F

fields (*dewetra3.serializers.AbstractComponentSerializer.Meta attribute*), 39
fields (*dewetra3.serializers.AbstractNameIconSerializer.Meta attribute*), 40
fields (*dewetra3.serializers.EventDataSerializer.Meta attribute*), 40
fields (*dewetra3.serializers.GeoScaleSerializer.Meta attribute*), 40
fields (*dewetra3.serializers.LayerCategorySerializer.Meta attribute*), 41
fields (*dewetra3.serializers.LayerSerializer.Meta attribute*), 41
fields (*dewetra3.serializers.PathHierarchySerializer.Meta attribute*), 42
fields (*dewetra3.serializers.ToolSerializer.Meta attribute*), 43
FilterByCategory (*class in dewetra3.admin*), 14
FilterByLayerManager (*class in dewetra3.admin*), 14
floodproof() (*dewetra3.dds.views.DDSDynamicLayer method*), 5
floodproofCompatibles() (*dewetra3.dds.views.DDSDynamicLayer method*), 5

formfield_overrides (*dewetra3.admin.AppAdmin attribute*), 14
formfield_overrides (*dewetra3.admin.LayerAdmin attribute*), 15

G

GeoScale (*class in dewetra3.models*), 20
geoscale (*dewetra3.models.Layer attribute*), 26
GeoScale.DoesNotExist, 20
GeoScale.MultipleObjectsReturned, 20
geoscale_id (*dewetra3.models.Layer attribute*), 26
GeoScaleSerializer (*class in dewetra3.serializers*), 40
GeoScaleSerializer.Meta (*class in dewetra3.serializers*), 40
get() (*dewetra3.views.CapabilitiesView method*), 44
get() (*dewetra3.views.ProxyView method*), 46
get_data() (*dewetra3.widget.widget.WidgetManager method*), 13
get_img() (*dewetra3.widget.widget.WidgetManager method*), 13
get_next_by_date() (*dewetra3.models.EventData method*), 19
get_previous_by_date() (*dewetra3.models.EventData method*), 19
get_queryset() (*dewetra3.admin.LayerAdmin method*), 15
get_queryset() (*dewetra3.views.LayerView method*), 45
get_queryset() (*dewetra3.views.ToolView method*), 48
get_url() (*dewetra3.serializers.ServerSerializer method*), 43

H

Hazard (*class in dewetra3.models*), 20
hazard (*dewetra3.models.EventData attribute*), 19
Hazard.DoesNotExist, 20
Hazard.MultipleObjectsReturned, 21
hazard_id (*dewetra3.models.EventData attribute*), 19
hazard_set (*dewetra3.models.Icon attribute*), 22
hazards (*dewetra3.models.Layer attribute*), 26
HazardSerializer (*class in dewetra3.serializers*), 41
HazardSerializer.Meta (*class in dewetra3.serializers*), 41
hierarchy (*dewetra3.models.Path attribute*), 32
hierarchy (*dewetra3.models.PathHierarchy attribute*), 33

I

Icon (*class in dewetra3.models*), 21
icon (*dewetra3.models.AbstractIconDescrModel attribute*), 16
icon (*dewetra3.models.DraCategory attribute*), 18
icon (*dewetra3.models.Hazard attribute*), 21
icon (*dewetra3.models.InspireTheme attribute*), 24
icon (*dewetra3.models.Layer attribute*), 26
icon (*dewetra3.models.Source attribute*), 36
icon (*dewetra3.models.Tag attribute*), 37
icon (*dewetra3.models.Tool attribute*), 38
Icon.DoesNotExist, 21
Icon.MultipleObjectsReturned, 21
icon_id (*dewetra3.models.AbstractIconDescrModel attribute*), 16
icon_id (*dewetra3.models.Tool attribute*), 38
id (*dewetra3.models.Chart attribute*), 17
id (*dewetra3.models.ChartManager attribute*), 17
id (*dewetra3.models.DraCategory attribute*), 18
id (*dewetra3.models.Event attribute*), 19
id (*dewetra3.models.EventData attribute*), 20
id (*dewetra3.models.GeoScale attribute*), 20
id (*dewetra3.models.Hazard attribute*), 21
id (*dewetra3.models.Icon attribute*), 22
id (*dewetra3.models.InfoManager attribute*), 23
id (*dewetra3.models.InspireTheme attribute*), 24
id (*dewetra3.models.Layer attribute*), 26
id (*dewetra3.models.LayerCategory attribute*), 30
id (*dewetra3.models.LayerManager attribute*), 30
id (*dewetra3.models.LegendManager attribute*), 31
id (*dewetra3.models.MapManager attribute*), 31
id (*dewetra3.models.MetadataManager attribute*), 32
id (*dewetra3.models.Path attribute*), 33
id (*dewetra3.models.PathHierarchy attribute*), 33
id (*dewetra3.models.PropertiesManager attribute*), 34
id (*dewetra3.models.Role attribute*), 34
id (*dewetra3.models.Serie attribute*), 35
id (*dewetra3.models.Server attribute*), 36
id (*dewetra3.models.Source attribute*), 36
id (*dewetra3.models.Tag attribute*), 37
id (*dewetra3.models.Tool attribute*), 38
id (*dewetra3.models.Widget attribute*), 38
InfoManager (*class in dewetra3.models*), 23
infomanager (*dewetra3.models.Layer attribute*), 26
InfoManager.DoesNotExist, 23
InfoManager.MultipleObjectsReturned, 23
infomanager_id (*dewetra3.models.Layer attribute*), 27
InfoManagerSerializer (*class in dewetra3.serializers*), 41
InfoManagerSerializer.Meta (*class in dewetra3.serializers*), 41
initial (*dewetra3.migrations.0001_initial.Migration attribute*), 1
InspireTheme (*class in dewetra3.models*), 24
InspireTheme.DoesNotExist, 24
InspireTheme.MultipleObjectsReturned, 24
inspiretheme_set (*dewetra3.models.Icon attribute*), 22
inspirethemes (*dewetra3.models.Layer attribute*), 27
IsEnabledFilter (*class in dewetra3.admin*), 15

IspireThemeSerializer (*class in dewetra3.serializers*), 41
IspireThemeSerializer.Meta (*class in dewetra3.serializers*), 41

L

lat (*dewetra3.models.EventData attribute*), 20
latn (*dewetra3.models.Layer attribute*), 27
lats (*dewetra3.models.Layer attribute*), 27
Layer (*class in dewetra3.models*), 24
layer (*dewetra3.models.Widget attribute*), 39
Layer.DoesNotExist, 24
Layer.MultipleObjectsReturned, 25
layer_id (*dewetra3.models.Widget attribute*), 39
layer_set (*dewetra3.models.ChartManager attribute*), 17
layer_set (*dewetra3.models.DraCategory attribute*), 18
layer_set (*dewetra3.models.EventData attribute*), 20
layer_set (*dewetra3.models.GeoScale attribute*), 20
layer_set (*dewetra3.models.Hazard attribute*), 21
layer_set (*dewetra3.models.Icon attribute*), 22
layer_set (*dewetra3.models.InfoManager attribute*), 23
layer_set (*dewetra3.models.InspireTheme attribute*), 24
layer_set (*dewetra3.models.LayerManager attribute*), 31
layer_set (*dewetra3.models.LegendManager attribute*), 31
layer_set (*dewetra3.models.MapManager attribute*), 32
layer_set (*dewetra3.models.MetadataManager attribute*), 32
layer_set (*dewetra3.models.PathHierarchy attribute*), 33
layer_set (*dewetra3.models.PropertiesManager attribute*), 34
layer_set (*dewetra3.models.Server attribute*), 36
layer_set (*dewetra3.models.Source attribute*), 37
layer_set (*dewetra3.models.Tag attribute*), 37
LayerAdmin (*class in dewetra3.admin*), 15
LayerCategory (*class in dewetra3.models*), 30
LayerCategory.DoesNotExist, 30
LayerCategory.MultipleObjectsReturned, 30
LayerCategorySerializer (*class in dewetra3.serializers*), 41
LayerCategorySerializer.Meta (*class in dewetra3.serializers*), 41
LayerCategoryView (*class in dewetra3.views*), 44
LayerManager (*class in dewetra3.models*), 30
layermanager (*dewetra3.models.Layer attribute*), 27
LayerManager.DoesNotExist, 30
LayerManager.MultipleObjectsReturned, 30
layermanager_id (*dewetra3.models.Layer attribute*), 27
LayerManagerSerializer (*class in dewetra3.serializers*), 41
LayerManagerSerializer.Meta (*class in dewetra3.serializers*), 41
layers (*dewetra3.models.LayerCategory attribute*), 30
layers (*dewetra3.models.Role attribute*), 34
LayerSerializer (*class in dewetra3.serializers*), 41
LayerSerializer.Meta (*class in dewetra3.serializers*), 41
layerstyle() (*dewetra3.dds.views.DDSDynamicLayer method*), 6
LayerView (*class in dewetra3.views*), 45
LegendManager (*class in dewetra3.models*), 31
legendmanager (*dewetra3.models.Layer attribute*), 27
LegendManager.DoesNotExist, 31
LegendManager.MultipleObjectsReturned, 31
legendmanager_id (*dewetra3.models.Layer attribute*), 27
LegendManagerSerializer (*class in dewetra3.serializers*), 42
LegendManagerSerializer.Meta (*class in dewetra3.serializers*), 42
list() (*dewetra3.dds.views.DDSDynamicLayer method*), 6
list_display (*dewetra3.admin.LayerAdmin attribute*), 15
list_filter (*dewetra3.admin.LayerAdmin attribute*), 15
list_max_show_all (*dewetra3.admin.AppAdmin attribute*), 14
list_max_show_all (*dewetra3.admin.LayerAdmin attribute*), 15
list_per_page (*dewetra3.admin.AppAdmin attribute*), 14
list_per_page (*dewetra3.admin.LayerAdmin attribute*), 15
lon (*dewetra3.models.EventData attribute*), 20
lone (*dewetra3.models.Layer attribute*), 27
lonw (*dewetra3.models.Layer attribute*), 27
lookups() (*dewetra3.admin.FilterByCategory method*), 14
lookups() (*dewetra3.admin.FilterByLayerManager method*), 14
lookups() (*dewetra3.admin.IsEnabledFilter method*), 15

M

main() (*in module manage*), 51
manage
    module, 51
MapManager (*class in dewetra3.models*), 31
mapmanager (*dewetra3.models.Layer attribute*), 28
MapManager.DoesNotExist, 31
MapManager.MultipleObjectsReturned, 31

mapmanager_id (*dewetra3.models.Layer attribute*), 28
MapManagerSerializer (*class in dewetra3.serializers*), 42
MapManagerSerializer.Meta (*class in dewetra3.serializers*), 42
media (*dewetra3.admin.AppAdmin property*), 14
media (*dewetra3.admin.LayerAdmin property*), 15
MetadataManager (*class in dewetra3.models*), 32
metadatamanager (*dewetra3.models.Layer attribute*), 28
MetadataManager.DoesNotExist, 32
MetadataManager.MultipleObjectsReturned, 32
metadatamanager_id (*dewetra3.models.Layer attribute*), 28
MetadataManagerSerializer (*class in dewetra3.serializers*), 42
MetadataManagerSerializer.Meta (*class in dewetra3.serializers*), 42
Migration (*class in dewetra3.migrations.0001_initial*), 1
Migration (*class in dewetra3.migrations.0002_layer_enabled*), 3
Migration (*class in dewetra3.migrations.0003_auto_20240213_1503*), 3
Migration (*class in dewetra3.migrations.0004_alter_server_user*), 3
Migration (*class in dewetra3.migrations.0005_alter_server_password*), 3
Migration (*class in dewetra3.migrations.0006_auto_20240227_0944*), 4
Migration (*class in dewetra3.migrations.0007_tool*), 4
Migration (*class in dewetra3.migrations.0008_tool_icon*), 4
mode (*dewetra3.models.Widget attribute*), 39
model (*dewetra3.serializers.ChartManagerSerializer.Meta attribute*), 40
model (*dewetra3.serializers.DraCategorySerializer.Meta attribute*), 40
model (*dewetra3.serializers.EventDataSerializer.Meta attribute*), 40
model (*dewetra3.serializers.EventSerializer.Meta attribute*), 40
model (*dewetra3.serializers.GeoScaleSerializer.Meta attribute*), 40
model (*dewetra3.serializers.HazardSerializer.Meta attribute*), 41
model (*dewetra3.serializers.InfoManagerSerializer.Meta attribute*), 41
model (*dewetra3.serializers.IspireThemeSerializer.Meta attribute*), 41
model (*dewetra3.serializers.LayerCategorySerializer.Meta attribute*), 41
model (*dewetra3.serializers.LayerManagerSerializer.Meta attribute*), 41
model (*dewetra3.serializers.LayerSerializer.Meta attribute*), 41
model (*dewetra3.serializers.LegendManagerSerializer.Meta attribute*), 42
model (*dewetra3.serializers.MapManagerSerializer.Meta attribute*), 42
model (*dewetra3.serializers.MetadataManagerSerializer.Meta attribute*), 42
model (*dewetra3.serializers.PathHierarchySerializer.Meta attribute*), 42
model (*dewetra3.serializers.PropertiesManagerSerializer.Meta attribute*), 42
model (*dewetra3.serializers.ServerSerializer.Meta attribute*), 42
model (*dewetra3.serializers.SourceSerializer.Meta attribute*), 43
model (*dewetra3.serializers.TagSerializer.Meta attribute*), 43
model (*dewetra3.serializers.ToolSerializer.Meta attribute*), 43
module
- dewetra3, 49
- dewetra3.admin, 14
- dewetra3.asgi, 15
- dewetra3.charts, 12
- dewetra3.charts.views, 10
- dewetra3.dds, 8
- dewetra3.dds.views, 5
- dewetra3.migrations, 5
- dewetra3.migrations.0001_initial, 1
- dewetra3.migrations.0002_layer_enabled, 3
- dewetra3.migrations.0003_auto_20240213_1503, 3
- dewetra3.migrations.0004_alter_server_user, 3
- dewetra3.migrations.0005_alter_server_password, 3
- dewetra3.migrations.0006_auto_20240227_0944, 4
- dewetra3.migrations.0007_tool, 4
- dewetra3.migrations.0008_tool_icon, 4
- dewetra3.models, 16
- dewetra3.sentinel, 9
- dewetra3.sentinel.views, 8
- dewetra3.serializers, 39
- dewetra3.settings, 43
- dewetra3.test, 12
- dewetra3.tools, 10
- dewetra3.tools.views, 9
- dewetra3.urls, 43
- dewetra3.views, 44

dewetra3.widget, 14
dewetra3.widget.service, 12
dewetra3.widget.tests, 12
dewetra3.widget.views, 12
dewetra3.widget.widget, 13
dewetra3.wsgi, 49
manage, 51
version, 53

N

name (*dewetra3.charts.views.ChartViewSet attribute*), 11
name (*dewetra3.dds.views.DDSDynamicLayer attribute*), 6
name (*dewetra3.models.AbstractNameModel attribute*), 16
name (*dewetra3.models.Widget attribute*), 39
name (*dewetra3.sentinel.views.SentinelLayer attribute*), 8
name (*dewetra3.tools.views.ToolsViewsSet attribute*), 10
name (*dewetra3.views.LayerCategoryView attribute*), 45
name (*dewetra3.views.LayerView attribute*), 45
name (*dewetra3.views.ServerView attribute*), 47
name (*dewetra3.views.TagView attribute*), 47
name (*dewetra3.views.ToolView attribute*), 48
name (*dewetra3.widget.views.WidgetDataView attribute*), 12

O

objects (*dewetra3.models.Chart attribute*), 17
objects (*dewetra3.models.ChartManager attribute*), 18
objects (*dewetra3.models.DraCategory attribute*), 18
objects (*dewetra3.models.Event attribute*), 19
objects (*dewetra3.models.EventData attribute*), 20
objects (*dewetra3.models.GeoScale attribute*), 20
objects (*dewetra3.models.Hazard attribute*), 21
objects (*dewetra3.models.Icon attribute*), 22
objects (*dewetra3.models.InfoManager attribute*), 24
objects (*dewetra3.models.InspireTheme attribute*), 24
objects (*dewetra3.models.Layer attribute*), 28
objects (*dewetra3.models.LayerCategory attribute*), 30
objects (*dewetra3.models.LayerManager attribute*), 31
objects (*dewetra3.models.LegendManager attribute*), 31
objects (*dewetra3.models.MapManager attribute*), 32
objects (*dewetra3.models.MetadataManager attribute*), 32
objects (*dewetra3.models.Path attribute*), 33
objects (*dewetra3.models.PathHierarchy attribute*), 33
objects (*dewetra3.models.PropertiesManager attribute*), 34
objects (*dewetra3.models.Role attribute*), 34
objects (*dewetra3.models.Serie attribute*), 35
objects (*dewetra3.models.Server attribute*), 36
objects (*dewetra3.models.Source attribute*), 37
objects (*dewetra3.models.Tag attribute*), 37
objects (*dewetra3.models.Tool attribute*), 38
objects (*dewetra3.models.Widget attribute*), 39
on_close() (*dewetra3.widget.service.WidgetService method*), 12
on_error() (*dewetra3.widget.service.WidgetService method*), 12
on_message() (*dewetra3.widget.service.WidgetService method*), 12
operations (*dewetra3.migrations.0001_initial.Migration attribute*), 1
operations (*dewetra3.migrations.0002_layer_enabled.Migration attribute*), 3
operations (*dewetra3.migrations.0003_auto_20240213_1503.Migration attribute*), 3
operations (*dewetra3.migrations.0004_alter_server_user.Migration attribute*), 3
operations (*dewetra3.migrations.0005_alter_server_password.Migration attribute*), 3
operations (*dewetra3.migrations.0006_auto_20240227_0944.Migration attribute*), 4
operations (*dewetra3.migrations.0007_tool.Migration attribute*), 4
operations (*dewetra3.migrations.0008_tool_icon.Migration attribute*), 4
options() (*dewetra3.views.CapabilitiesView method*), 44
options() (*dewetra3.views.ProxyView method*), 46

P

parameter_name (*dewetra3.admin.FilterByCategory attribute*), 14
parameter_name (*dewetra3.admin.FilterByLayerManager attribute*), 14
parameter_name (*dewetra3.admin.IsEnabledFilter attribute*), 15
password (*dewetra3.models.Server attribute*), 36
Path (*class in dewetra3.models*), 32
path (*dewetra3.models.PathHierarchy attribute*), 33
Path.DoesNotExist, 32
Path.MultipleObjectsReturned, 32
path_id (*dewetra3.models.PathHierarchy attribute*), 33
PathHierarchy (*class in dewetra3.models*), 33
PathHierarchy.DoesNotExist, 33
PathHierarchy.MultipleObjectsReturned, 33
PathHierarchySerializer (*class in dewetra3.serializers*), 42
PathHierarchySerializer.Meta (*class in dewetra3.serializers*), 42
paths (*dewetra3.models.Layer attribute*), 28
period (*dewetra3.models.Widget attribute*), 39
permission_classes (*dewetra3.views.LayerView attribute*), 45

permission_classes (*dewetra3.widget.views.WidgetDataView attribute*), 12
post() (*dewetra3.views.ProxyView method*), 46
print() (*dewetra3.tools.views.ToolsViewsSet method*), 10
properties (*dewetra3.models.Widget attribute*), 39
properties() (*dewetra3.dds.views.DDSDynamicLayer method*), 6
PropertiesManager (*class in dewetra3.models*), 33
propertiesmanager (*dewetra3.models.Layer attribute*), 28
PropertiesManager.DoesNotExist, 34
PropertiesManager.MultipleObjectsReturned, 34
propertiesmanager_id (*dewetra3.models.Layer attribute*), 28
PropertiesManagerSerializer (*class in dewetra3.serializers*), 42
PropertiesManagerSerializer.Meta (*class in dewetra3.serializers*), 42
props (*dewetra3.models.AbstractWithComponentModel attribute*), 16
props (*dewetra3.models.AbstractWithPropertiesModel attribute*), 17
proxy() (*in module dewetra3.views*), 48
ProxyView (*class in dewetra3.views*), 45
publish() (*dewetra3.dds.views.DDSDynamicLayer method*), 6
publish_default() (*dewetra3.dds.views.DDSDynamicLayer method*), 6
punctualserie() (*dewetra3.dds.views.DDSDynamicLayer method*), 7

Q

queryset (*dewetra3.views.LayerCategoryView attribute*), 45
queryset (*dewetra3.views.LayerView attribute*), 45
queryset (*dewetra3.views.ServerView attribute*), 47
queryset (*dewetra3.views.TagView attribute*), 47
queryset (*dewetra3.views.ToolView attribute*), 47, 48
queryset() (*dewetra3.admin.FilterByCategory method*), 14
queryset() (*dewetra3.admin.FilterByLayerManager method*), 14
queryset() (*dewetra3.admin.IsEnabledFilter method*), 15

R

related (*dewetra3.models.Layer attribute*), 28
republish() (*dewetra3.dds.views.DDSDynamicLayer method*), 7
Role (*class in dewetra3.models*), 34
Role.DoesNotExist, 34
Role.MultipleObjectsReturned, 34
roles (*dewetra3.models.Layer attribute*), 29
roles (*dewetra3.models.Serie attribute*), 35
roles (*dewetra3.models.Tool attribute*), 38
run() (*dewetra3.widget.service.WidgetService method*), 12

S

save_as (*dewetra3.admin.AppAdmin attribute*), 14
save_as (*dewetra3.admin.LayerAdmin attribute*), 15
search_fields (*dewetra3.admin.AppAdmin attribute*), 14
search_fields (*dewetra3.admin.LayerAdmin attribute*), 15
select_parser() (*dewetra3.views.BypassFormatArgumentNegotiation method*), 44
select_renderer() (*dewetra3.views.BypassFormatArgumentNegotiation method*), 44
sensor_info() (*dewetra3.sentinel.views.SentinelLayer method*), 8
sensors() (*dewetra3.sentinel.views.SentinelLayer method*), 9
SentinelLayer (*class in dewetra3.sentinel.views*), 8
serializer_class (*dewetra3.views.LayerCategoryView attribute*), 45
serializer_class (*dewetra3.views.LayerView attribute*), 45
serializer_class (*dewetra3.views.ServerView attribute*), 47
serializer_class (*dewetra3.views.TagView attribute*), 47
serializer_class (*dewetra3.views.ToolView attribute*), 47, 48
Serie (*class in dewetra3.models*), 35
Serie.DoesNotExist, 35
Serie.MultipleObjectsReturned, 35
series (*dewetra3.models.Role attribute*), 34
Server (*class in dewetra3.models*), 35
server (*dewetra3.models.Layer attribute*), 29
Server.DoesNotExist, 35
Server.MultipleObjectsReturned, 35
server_id (*dewetra3.models.Layer attribute*), 29
ServerSerializer (*class in dewetra3.serializers*), 42
ServerSerializer.Meta (*class in dewetra3.serializers*), 42
ServerView (*class in dewetra3.views*), 46
Source (*class in dewetra3.models*), 36
source (*dewetra3.models.Layer attribute*), 29
Source.DoesNotExist, 36
Source.MultipleObjectsReturned, 36

source_id (*dewetra3.models.Layer attribute*), 29
source_set (*dewetra3.models.Icon attribute*), 22
SourceSerializer (*class in dewetra3.serializers*), 43
SourceSerializer.Meta (*class in dewetra3.serializers*), 43
stations_sensors() (*dewetra3.sentinel.views.SentinelLayer method*), 9
suffix (*dewetra3.charts.views.ChartViewSet attribute*), 12
suffix (*dewetra3.dds.views.DDSDynamicLayer attribute*), 7
suffix (*dewetra3.sentinel.views.SentinelLayer attribute*), 9
suffix (*dewetra3.tools.views.ToolsViewsSet attribute*), 10
suffix (*dewetra3.views.LayerCategoryView attribute*), 45
suffix (*dewetra3.views.LayerView attribute*), 45
suffix (*dewetra3.views.ServerView attribute*), 47
suffix (*dewetra3.views.TagView attribute*), 47
suffix (*dewetra3.views.ToolView attribute*), 48
suffix (*dewetra3.widget.views.WidgetDataView attribute*), 13

T

Tag (*class in dewetra3.models*), 37
Tag.DoesNotExist, 37
Tag.MultipleObjectsReturned, 37
tag_set (*dewetra3.models.Icon attribute*), 23
tags (*dewetra3.models.Layer attribute*), 29
TagSerializer (*class in dewetra3.serializers*), 43
TagSerializer.Meta (*class in dewetra3.serializers*), 43
TagView (*class in dewetra3.views*), 47
thumb (*dewetra3.models.Layer attribute*), 30
title (*dewetra3.admin.FilterByCategory attribute*), 14
title (*dewetra3.admin.FilterByLayerManager attribute*), 14
title (*dewetra3.admin.IsEnabledFilter attribute*), 15
Tool (*class in dewetra3.models*), 37
Tool.DoesNotExist, 38
Tool.MultipleObjectsReturned, 38
tool_set (*dewetra3.models.Icon attribute*), 23
tools (*dewetra3.models.Role attribute*), 35
ToolSerializer (*class in dewetra3.serializers*), 43
ToolSerializer.Meta (*class in dewetra3.serializers*), 43
ToolsViewsSet (*class in dewetra3.tools.views*), 9
ToolView (*class in dewetra3.views*), 47
type (*dewetra3.models.Chart attribute*), 17

U

update() (*dewetra3.widget.widget.WidgetManager method*), 13
upload_image() (*dewetra3.tools.views.ToolsViewsSet method*), 10
url (*dewetra3.models.Server attribute*), 36
use_proxy (*dewetra3.models.Server attribute*), 36
user (*dewetra3.models.Server attribute*), 36

V

value (*dewetra3.models.Icon attribute*), 23
variables() (*dewetra3.dds.views.DDSDynamicLayer method*), 7
version
    module, 53

W

Widget (*class in dewetra3.models*), 38
Widget.DoesNotExist, 38
Widget.MultipleObjectsReturned, 38
widget_data() (*dewetra3.widget.views.WidgetDataView method*), 13
widget_img() (*dewetra3.widget.views.WidgetDataView method*), 13
widget_set (*dewetra3.models.Layer attribute*), 30
WidgetDataView (*class in dewetra3.widget.views*), 12
WidgetManager (*class in dewetra3.widget.widget*), 13
WidgetService (*class in dewetra3.widget.service*), 12
wmsbaselayer (*dewetra3.models.Widget attribute*), 39
wmsparams (*dewetra3.models.Widget attribute*), 39